

Technical Report on LMS-AR Memory Bandwidth Regulation

Source: github.com/ss22ongithub/LMS-AdaptiveRegulator wiki

Author: Sudarshan S · Last edited: May 23, 2026

This Technical Report on LMS-AR regulation scheme is intended for providing additional information.

I. INTRODUCTION

The problem of providing temporal isolation in embedded systems with shared DRAM as one of the contention points has been studied from both hardware [1] and software [2] design perspectives. Shared DRAM contention undermines timing predictability in multi-core platforms. When multiple cores simultaneously access shared memory, they compete for DRAM banks, row buffers, and memory controller bandwidth, creating complex interference patterns where a memory request from one core can delay requests from other cores by orders of magnitude depending on bank conflicts, row buffer misses, and refresh cycles.

II. MOTIVATION

The out-of-core approach used in Mempol [3] requires a separate processing element (on board micro-controller) with its own SRAM to operate. This element controls the bandwidth consumed by the other CPU cores called the regulated cores via on board debug interfaces. In off-the-shelf platforms, we do not have this option, and the regulator module has to execute as part of the master core (Core-0). This also causes the master core to generate its own memory transactions, which is a potential source of interference. In this work, we proceed with the premise that if the cores on the system are operated close to the sustainable bandwidth [4] while processing the workloads in such a way that the workload requirements are satisfied, then the workload would experience only a very minimal interference.

In order to validate the premise, we consider two benchmarks from SPEC2017 as the workload (one foreground (FG) task and one background (BG) task), which are executed on the cores in an isolated manner. The benchmarks were profiled a priori measuring their memory bandwidth consumption to understand if they are memory heavy in execution. We use the metric instructions-per-cycle (IPC) via the `perf` tool to compare the performance. The BG benchmark is memory heavy in comparison to the FG benchmark, and hence it depicts lower IPC. When the workloads are co-scheduled on different cores (without any regulation), we observe that the workloads suffer a drop in the IPC (instruction-per-cycle) values, as shown in Fig. 1. Subsequently, we ensure the cores are allocated certain predefined budget of memory bandwidth for a certain interval and replenished every cycle. This is done by making use of the performance monitoring counters (PMC). The core is throttled if it exhausts the budget within the period T . By statically allocating certain fixed bandwidth budget among the cores and throttling the cores on exhaustion we find that the workloads performed better, which is indicated by the higher IPC values (solid lines in Fig. 1) compared to the earlier scenario where workloads were allowed to run without any regulation or throttling (dotted lines in Fig. 1). The primary reason for this behavior is that the workloads experience a contention at the DRAM. A simple static allocation and throttling of cores reduces the possibilities of experiencing such contention at the DRAM controller. We make the following contributions in this work:

1. We propose a novel kernel space memory bandwidth regulation mechanism LMS-AR: LMS Prediction-based Adaptive Regulator, which can periodically estimate the memory bandwidth requirement of the workloads running, by using an adaptive filtering technique.
2. We implement a scheme to regulate the memory bandwidth allocation, outside of the cores as a loadable Linux kernel module and deploy it on COTS hardware.
3. We execute various types of workloads using SPEC2017 to evaluate the performance of the regulation scheme.

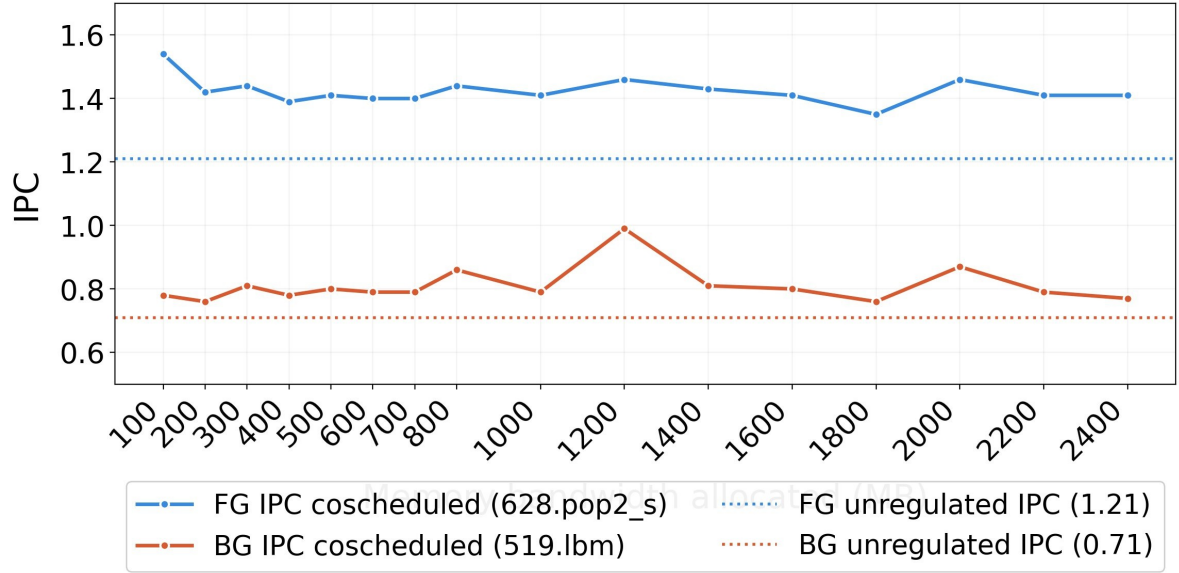


Figure 1. Comparison of IPC performance: Unregulated vs throttled with fixed bandwidth budget. X-axis: allocated memory bandwidth budget (in MB/s).

III. SYSTEM OVERVIEW

The primary components of the system architecture are illustrated in Fig. 2. The hardware setup consists of a multi core platform, which interfaces with a shared DRAM. The platform used supports a performance monitoring unit (PMU) for each CPU core, that helps to track the Last Level Cache (LLC) misses and write-backs due to load or store instructions. The core that runs the regulator is termed as the *master core* and the ones which are throttled are called the *regulated cores*. The proposed LMS-AR memory bandwidth regulator resides as a module within the kernel space (shown as dotted rectangle in Fig. 2). It contains a bandwidth estimator and a bandwidth regulator that is exclusively executed on the master Core (Core 0). The regulator does not maintain a timer, however it periodically wakes up to perform estimation and replenishment of the bandwidth for each core. This interval between successive resource replenishments is treated as the *regulation period T* . The bandwidth estimator analyzes the memory bandwidth consumed by each CPU core during the past regulation periods and estimates the requirement for the upcoming regulation interval. The bandwidth regulator is responsible to 1) replenish the bandwidth to each CPU core using a resource allocation scheme, and 2) un-throttle the CPU core at start of the regulation interval if the core is in throttled state. The regulator ensures that bandwidth consumed does not exceed a predefined bandwidth allocation for the system as a whole. Additionally, each core is associated with 1) a high priority task which acts as throttler that suspends the core and thereby blocks execution of tasks in that core, in turn blocking the memory transaction originating from the core, and 2) an interrupt handler which is triggered when the bandwidth budget is exhausted.

The CPU generates a lot of read and write memory transactions typically known as the load and store instructions. Upon a load or store instruction resulting in a cache miss, a read transaction is issued to retrieve the corresponding cache block from main memory. Should the evicted line carry dirty data, it is flushed to memory through a write transaction known as a cache write-back. The total number of memory accesses incurred by a core can be approximated by summing LLC misses and write-back transactions. Hence, the estimator keeps track of the number of LLC cache misses and write-backs generated by each CPU core that is available from Performance Monitoring Unit (PMU) in order to compute the future requirements for bandwidth.

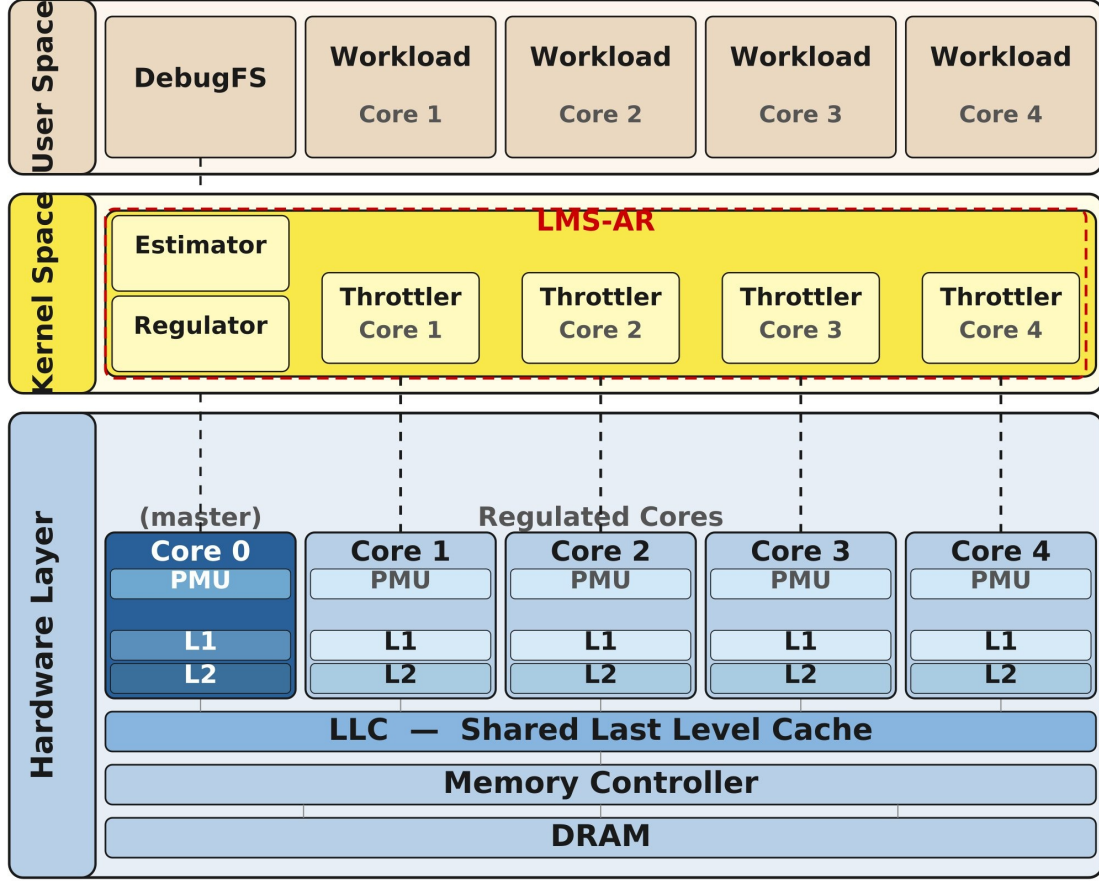


Figure 2. System Overview: LMS-AR's estimator-regulator components execute in kernel space, entirely running on Core 0 (master core).

IV. LMS-AR: LMS PREDICTION-BASED ADAPTIVE REGULATOR

Our proposed LMS-AR regulation mechanism consists of a LMS Prediction-based bandwidth estimator, an adaptive regulator and the necessary logic to throttle the core. These proposed blocks are explained in detail in the following subsections.

A. LMS based Estimator

The memory bandwidth demands of workloads running on individual cores are inherently dynamic and lack discernible patterns that could be reliably exploited for workload-level estimation. As a result, the predictor must meet three key requirements: (1) it must accurately forecast bandwidth demand for the upcoming regulation interval; (2) it must respond quickly to fluctuations in bandwidth consumption; and (3) it must be simple enough to implement within a Linux kernel module with acceptable runtime overhead.

These requirements make the Linear Least Mean Square (LMS) algorithm a natural fit for the prediction model. The LMS algorithm [5] is a well-established adaptive filtering technique that iteratively refines filter coefficients to minimize the mean square error between a desired signal and the filter output. Its low computational complexity and ease of implementation make it attractive for a wide range of prediction and time-series forecasting tasks.

The prediction model maintains a per-core history of observed memory bandwidth and estimates the bandwidth requirement for the next regulation interval as a weighted linear combination of past observations. The predictor weights are learned online through recursive updates performed at the end of each regulation period. Specifically, we employ the LMS predictor to forecast the memory bandwidth demand of each CPU core based on its consumption over preceding regulation intervals.

Let $b_c[t]$ denote the measured bandwidth of core c at interval t . The predicted bandwidth at interval $(t + 1)$ is

expressed as:

$$\hat{b}_c[t+1] = \sum_{i=0}^{N-1} w_i[t] x[t-i] = W[t]^T X[t] \quad (1)$$

where N is the number of historical samples retained, and $w_i[t]$ for $i = 0, \dots, N-1$ are the predictor weight coefficients, which are unknown at the start of each prediction cycle. The weight vector and input vector are defined as:

$$W[t] = [w_0, w_1, \dots, w_{N-1}]^T \quad (2)$$

$$X[t] = [x[t], x[t-1], \dots, x[t-N+1]]^T \quad (3)$$

The weights are adapted to minimize the prediction error $e[t]$, defined as the difference between the true and predicted bandwidth values:

$$e[t] = b_c[t+1] - \hat{b}_c[t+1] \quad (4)$$

Substituting Equation (1) into Equation (4) yields:

$$e[t] = b_c[t+1] - W[t]^T X[t] \quad (5)$$

The weight vector is initialized to a small value and updated at the end of each regulation interval according to the standard LMS update rule:

$$W[t+1] = W[t] + \eta e[t] X[t] \quad (6)$$

where η is the learning rate. To reduce sensitivity to the choice of η , a normalized variant of Equation (6) is adopted, in which the step size is scaled by the energy of the input vector.

C. Regulator

The regulation-estimation scheme is described in **Algorithm 1**. At every regulation interval, the master core determines the regulated core's consumed read and write bandwidths B_c^r and B_c^w and appends them to per-core circular history buffers H_c^r and H_c^w , respectively. The LMS predictor is then invoked per core and per traffic class (i.e., reads and writes separately), yielding $\hat{B}_c^r$ and $\hat{B}_c^w$ from the weight vectors W_c^r and W_c^w . If $\hat{B}_c^{total} = \hat{B}_c^r + \hat{B}_c^w$ fits within the per-core ceiling B_c^{max} , the full predicted amounts are allocated; otherwise, the ceiling is split proportionally, with any surplus returned to a global pool B_{pool} available to other cores. The allocations are programmed into the PMU period registers, counters are restarted, and the core is un-throttled. At the end of the interval, per-class prediction errors update W_c^r and W_c^w via the LMS rule described in Section III. A.

The per-core ceiling B_c^{max} is the maximum bandwidth a CPU core is allowed to consume by the regulator. This is statically allocated number which can be decided after performing a sustainable bandwidth assessment for a specific hardware platform.

Algorithm 1 LMS-AR Adaptive Regulation

Require: Per-core read/write PMU counters, circular history buffers H_c^r, H_c^w , LMS weight vectors W_c^r, W_c^w , per-core bandwidth limit B_c^{max} , global bandwidth pool B_{pool} , history window HIST_SIZE

```

1: while master thread is active do
2:   for each regulated core  $c$  do
     Bandwidth Measurement
3:     Read the PMU counters for core  $c$ 
4:     Determine  $B_c^r$  and  $B_c^w$  as difference
       between successive counter readings
     Bandwidth Prediction
5:      $H_c^r[i] \leftarrow B_c^r$ 
6:      $H_c^w[i] \leftarrow B_c^w$ 
7:      $\hat{B}_c^r \leftarrow W_c^{rT} \cdot H_c^r + B_c^{init}$ 
8:      $\hat{B}_c^w \leftarrow W_c^{wT} \cdot H_c^w + B_c^{init}$ 
     Bandwidth Allocation
9:      $\hat{B}_c^{total} \leftarrow \hat{B}_c^r + \hat{B}_c^w$ 
10:    if  $\hat{B}_c^{total} \leq B_c^{max}$  then
11:       $A_c^r \leftarrow \hat{B}_c^r, A_c^w \leftarrow \hat{B}_c^w$ 
12:    else
13:       $A_c^r \leftarrow B_c^{max} \cdot \frac{\hat{B}_c^r}{\hat{B}_c^{total}}, A_c^w \leftarrow B_c^{max} \cdot \frac{\hat{B}_c^w}{\hat{B}_c^{total}}$ 
14:    end if
15:     $B_{pool} += \max(0, B_c^{max} - (A_c^r + A_c^w))$ 
     Counter Replenish
16:    Program PMU counters  $\leftarrow A_c^r, A_c^w$ 
17:    Unthrottle core  $c$ 
18:    Restart PMU read and write counters
     LMS Weight Update
19:     $e_c^r \leftarrow B_c^r - \hat{B}_c^{r,prev}$ 
20:     $e_c^w \leftarrow B_c^w - \hat{B}_c^{w,prev}$ 
21:     $W_c^r \leftarrow W_c^r + \eta \cdot e_c^r \cdot H_c^r$ 
22:     $W_c^w \leftarrow W_c^w + \eta \cdot e_c^w \cdot H_c^w$ 
     Advance History Index
23:     $i \leftarrow (i + 1) \bmod \text{HIST\_SIZE}$ 
24:     $\hat{B}_c^{r,prev} \leftarrow \hat{B}_c^r, \hat{B}_c^{w,prev} \leftarrow \hat{B}_c^w$ 
25:   end for
26:   sleep  $T_{reg}$ 
27: end while

```

Algorithm 1. LMS-AR Adaptive Regulation (pseudocode).

D. Stalling the core

The functionality of stalling the core is an important step in preventing the core from consuming more memory bandwidth than what is allocated. In this work, a throttler thread similar to [2] is used and certain architecture specific instructions (such as MWAIT and MONITOR for x86) are executed indicating the core to suspend execution, which may hint the processor to move to low power states supported by the OS. The actual transition to and from low power state is dependent on the firmware configuration.

Using FPU Inside Linux Kernel

Estimating memory bandwidth requires floating-point arithmetic — specifically products and sums over measured values — to achieve a reasonably accurate result. However, floating-point computation is generally discouraged in kernel modules, as it introduces non-trivial overhead and risks unsafe handling of the Floating Point Unit (FPU) context. Here, we explicitly enable the floating-point unit within kernel space, carefully bracketing only the necessary calculations (Lines 21-22 Algorithm 1), within floating-point critical sections, in order to keep the implementation safe and lightweight while producing bandwidth estimates that closely track actual consumption. We employ generic runtime APIs (architecture specific implementations) provided in Linux kernel [6] to create these critical sections. These APIs typically create critical sections by ensuring 1) the preemption is disabled 2) sanity checks the context i.e., if FPU is usable at the point 3) saves the interrupted task's FPU state, if necessary and 4) initializes the control registers to a known value, in that order. The critical sections are free of blocking code or nested calls and use batched floating point computations.

These computations are exclusive only to FPU of the master core - one of the direct advantages of performing out-of-core regulation.

V. EXPERIMENTAL EVALUATIONS

Section A. Metrics

The IPC slowdown ratio (runalone IPC / co-scheduled IPC) [2], which indicates the extent to which co-scheduling multiple workloads on a shared memory system can degrade individual workload performance relative to running alone, is used as the primary metric to compare the performance of the regulators. A ratio of 1.0 indicates no degradation, while higher values reflect greater interference caused by contention on the shared memory bus.

Section B. Experimental Setup

The experiments were conducted on a desktop platform equipped with an Intel® Core™ i5-11400 processor (Rocket Lake), featuring 6 cores and 12 threads with a base frequency of 2.60 GHz. Though the system supports four DIMM slots, for our work it is equipped with a single DRAM module 3200 MT/s. For all the experiments we use 16 GB of dual-channel DDR4-3200 memory, providing a peak memory bandwidth of 50 GB/s. The processor supports hardware-accelerated floating-point operations. For the purpose of this work hyper-threading was disabled in the firmware to ensure the cores being used are actual physical cores. Hardware pre-fetcher was disabled using model specific registers [7] in order to avoid hiding the inherent memory latency between the CPU and DRAM. We use the stock linux kernel version 6.8.0 that comes along with Ubuntu 22.04 LTS. Additionally, cpusets infrastructure in Linux is used to isolate the CPU cores and selectively execute the desired SPEC-CPU2017 benchmark as a workload. Finally we use a regulation interval of 1 ms for all our experiments (Refer Section IV.G for explanation).

Section C. Sustainable Bandwidth Assessment

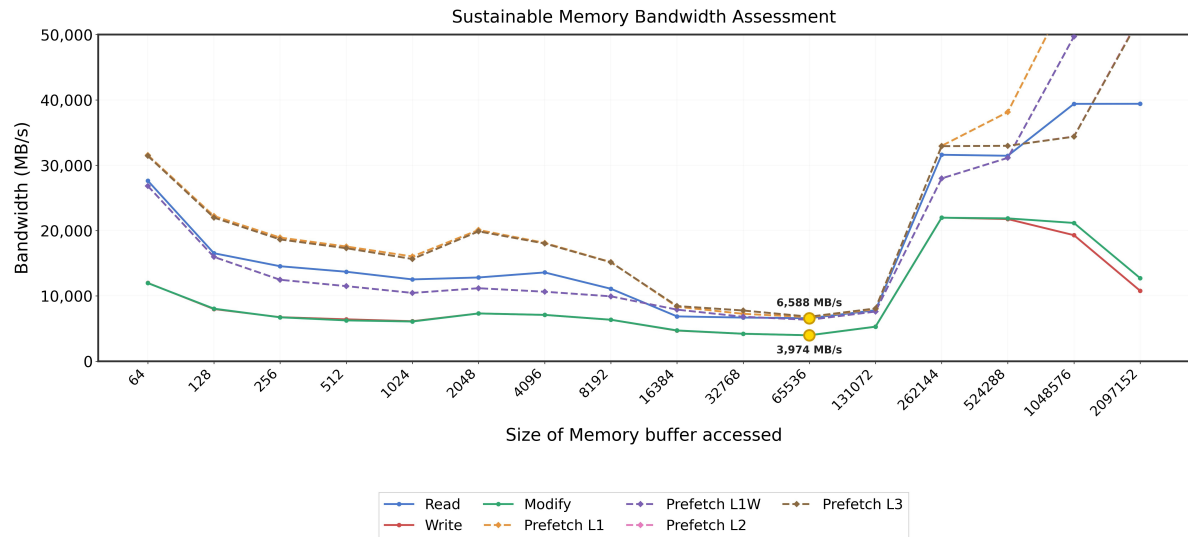
Sustainable bandwidth is the bandwidth the memory controller of a DRAM withstands without resulting in over saturation. Determining the sustainable bandwidth of the system is a significant step setting the limits of how much memory bandwidth a core or a set of cores can utilize in the system without leading to interference. In this context, we reuse publicly available *Bench* tool to perform, sustainable bandwidth measurements for our platform. Based on the measurement we fix the sustainable bandwidth at $B_{sustain}^r = 6000$ MB/s for reads and $B_{sustain}^w = 4000$ MB/s for writes on our setup. This Technical report provides further details on the assessment.

Memory access patterns start with relatively high bandwidth at small step sizes and show a significant drop as the step size increases beyond a certain threshold. This drop is a direct consequence of the CPU cache hierarchy — small step sizes fit within cache and enjoy high bandwidth, while larger step sizes start causing cache misses and force accesses to main memory.

The peak read bandwidth observed is close to 28000 MB/s because the hardware pre-fetcher can predict and stream sequential read patterns very efficiently into cache. The peak write bandwidth (23000 MB/s) is noticeably lower than read at small steps, which is expected since writes require a Read-for-Ownership (RFO) transaction before modifying a cache line. Modify bandwidth (which requires a read and a write operation to the same location) follows a very similar trend as the write bandwidth.

The bandwidth drop between 16384 and 65536 bytes corresponds to the transition from L2/L3 cache to main memory. Once the stride between accesses exceeds what the cache can hold, each access becomes a DRAM fetch, and bandwidth drops roughly 4 times. The pre-fetch lines (L1, L1W, L2, L3) track closely with each other and largely follow the same trend as the read bandwidth. At small step sizes they achieve very high bandwidth — often matching or slightly exceeding plain read — because explicit pre-fetch instructions keep the memory pipeline busy and reduce latency stalls. However, they experience the same sharp drop when the working set exceeds the cache capacity, collapsing to low DRAM-level bandwidth at large step sizes. For sizes higher than 1.3 MB, we see an increased performance possibly due to successful hit in LLC.

Based on this we fix a sustainable bandwidth for read as $B_{sustain}^r = 6000$ MB/s and sustainable bandwidth for write operation as $B_{sustain}^w = 4000$ MB/s for our setup.



Sustainable Memory Bandwidth Assessment using the Bench tool. Curves: Read, Write, Modify, Prefetch L1, Prefetch L1W, Prefetch L2, Prefetch L3. Sustainable points marked at 6,588 MB/s (read) and 3,974 MB/s (modify) at 65,536-byte buffer.

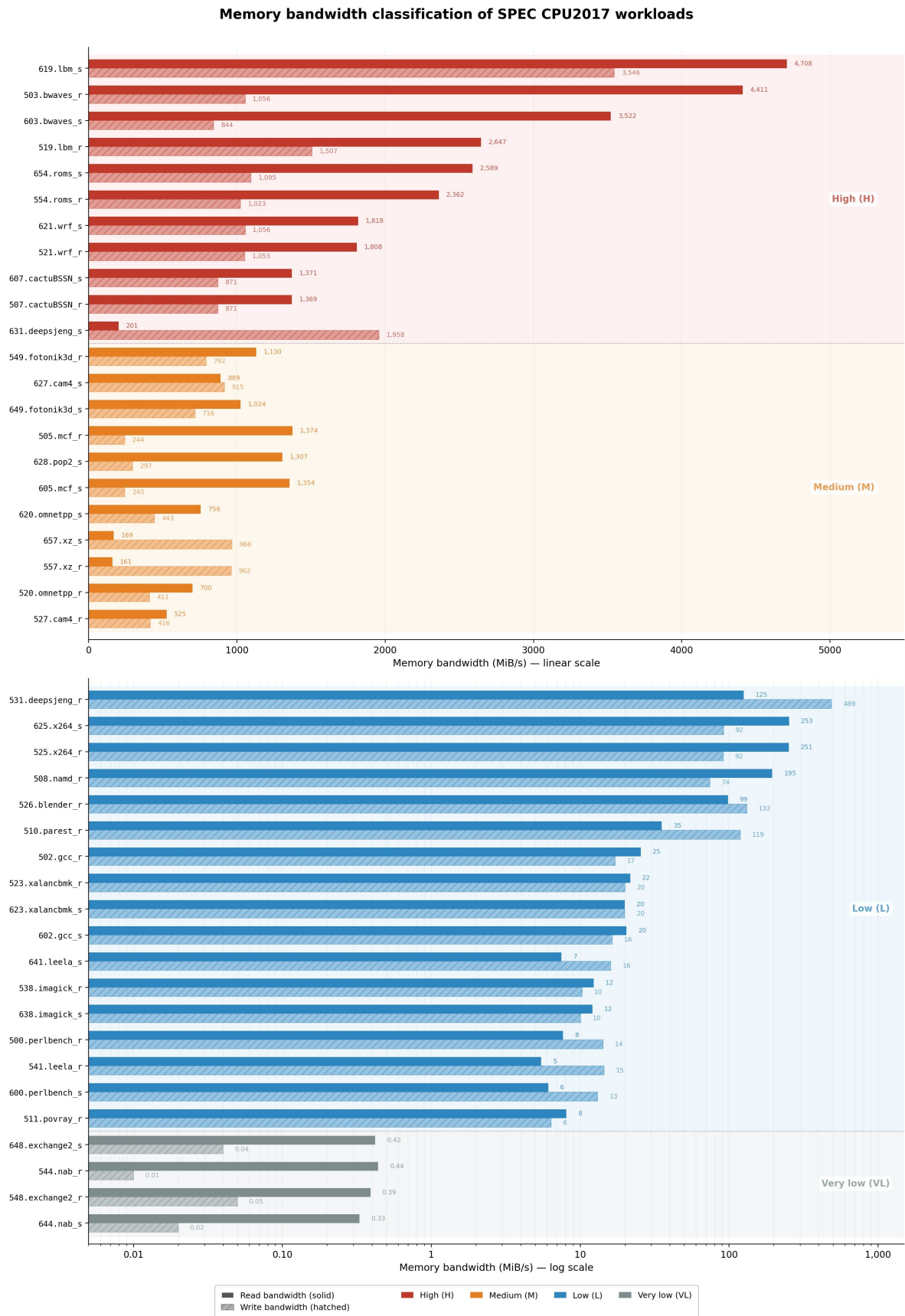
Section D. Workload Profiling

We compute the average bandwidth consumed for each workload to gain insights about the characteristic of the workload — whether they are read heavy or write heavy — using the `perf` tool. We additionally classify the workloads as High, Medium or Low to indicate its memory bandwidth usage intensity as shown in Table 1.

Workload Name	perf_read_bw (MiB/s)	perf_write_bw (MiB/s)	Sum BW (MiB/s)	Class
619.lbm_s	4707.96659	3545.85888	8253.82547	H
503.bwaves_r	4410.625	1056.159	5466.784	H
603.bwaves_s	3521.6737	843.7483	4365.422	H
519.lbm_r	2646.60588	1506.66	4153.26588	H
654.roms_s	2589.45333	1095.228666	3684.681996	H
554.roms_r	2362.41	1023.21428	3385.62428	H
621.wrf_s	1817.757	1056.12342	2873.88042	H
521.wrf_r	1807.8822	1052.73085	2860.61305	H
607.cactuBSSN_s	1370.77571	871.2328	2242.00851	H
507.cactuBSSN_r	1369.40785	871.0621	2240.46995	H
631.deepsjeng_s	200.879729	1957.96675	2158.846479	H
549.fotonik3d_r	1130.3657	791.94	1922.3057	M
627.cam4_s	888.5145	915.23636	1803.75086	M
649.fotonik3d_s	1023.923	716.45	1740.373	M
505.mcf_r	1374.3972	244.36625	1618.76345	M
628.pop2_s	1306.81625	296.7281	1603.54435	M
605.mcf_s	1353.90675	244.7515	1598.65825	M
620.omnetpp_s	755.5292	443.2646	1198.7938	M
657.xz_s	168.81368	966.368421	1135.182101	M
557.xz_r	160.6747	961.7139	1122.3886	M
520.omnetpp_r	700.0585	410.8635	1110.922	M
527.cam4_r	524.8903	416.48322	941.37352	M
531.deepsjeng_r	125.25879	488.9152	614.17399	L
625.x264_s	253.05012	91.96233	345.01245	L
525.x264_r	251.41831	91.7197	343.13801	L
508.namd_r	195.2099	74.38375	269.59365	L
526.blender_r	98.51	131.7666	230.2766	L
510.parest_r	35.3722	119.4259	154.7981	L
502.gcc_r	25.476	17.226	42.702	L
523.xalancbmk_r	21.72	20.06	41.78	L

623.xalancbmk_s	19.92	19.963333	39.883333	L
602.gcc_s	20.47666	16.463333	36.939993	L
641.leela_s	7.492	16.0685	23.5605	L
538.imagick_r	12.30666	10.31	22.61666	L
638.imagick_s	12.056666	10.10333	22.159996	L
500.perlbench_r	7.68	14.286	21.966	L
541.leela_r	5.466	14.507	19.973	L
600.perlbench_s	6.09705	13.13348	19.23053	L
511.povray_r	8.07499	6.3875	14.46249	L
544.nab_r	0.44	0.01	0.45	VL
548.exchange2_r	0.388666	0.04566	0.434326	VL
644.nab_s	0.334999	0.023	0.357999	VL
648.exchange2_s	0.415	0.0386666	0.4536666	VL

The Workload characteristics visualization for (log scale) Low and Very Low and (linear scale) for High and Medium memory bandwidth.



Memory bandwidth classification of SPEC CPU 2017 workloads. Read bandwidth (solid) and write bandwidth (hatched), grouped by intensity class: High (H), Medium (M), Low (L), Very Low (VL). Top panel uses a linear scale; bottom panel uses a log scale.

Section E. Evaluations on 2-core Setup

We create a setup where the estimation and regulation mechanism is executed on Master core (core 0) and the regulated cores are Core 1 and Core 3, designated as C1 and C3 respectively. The Cores C1 and C3 are

isolated using the cpusets facility provided in the Linux Kernel. The regulated cores are assigned a bandwidth limit of 1 GB/s each such that the total utilized bandwidth does not exceed the overall bandwidth allocated for the system.

We select 519.lbm as the background task, for all our experiments, due to its high memory intensity and ability to generate a sustained stream of read and write memory traffic, making it an ideal background workload for stressing the shared memory. The measurements are taken for fixed amount of time (20 s) for each set of co-scheduled tasks. Fig. 3 compares the IPC slowdown ratios of MemGuard and the LMS-AR across 22 of the 42 workloads of the SPEC CPU 2017 benchmarks. The improvements are observed across medium and low memory intensive workloads. Across the 22 SPEC CPU benchmarks in the 2-core setup, LMS-AR achieves an average IPC slowdown ratio of 1.0730 with a standard deviation of 0.0858, compared to MemGuard's average of 1.1331 with a standard deviation of 0.1223. For rest of the workloads (which are predominantly memory-intensive), slowdown ratios of < 1 are observed with LMS-AR as the high LLC misses seen in stand-alone measurements, are regulated efficiently by LMS-AR leading to higher IPC than stand-alone IPC. This behavior is further corroborated by the Table in Section E of the Technical Report on LMS-AR [9].

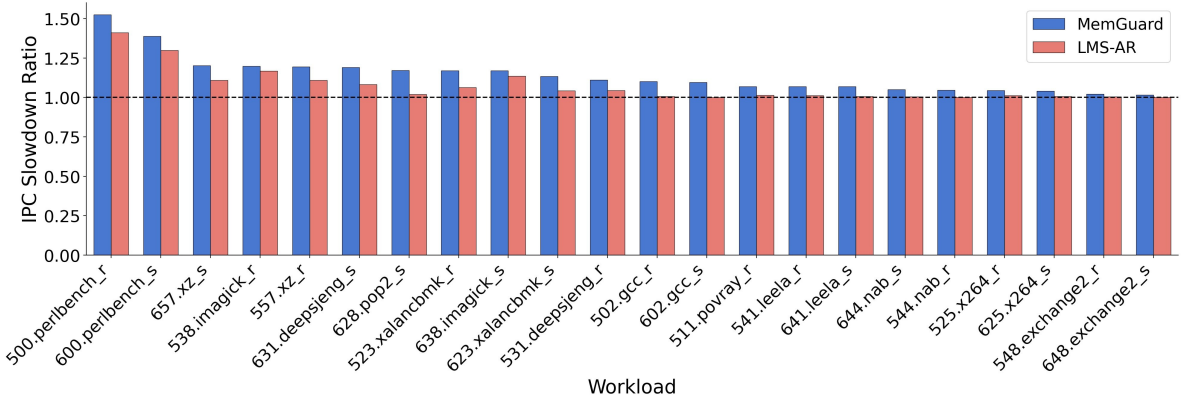


Figure 3. Comparison of slowdown ratios for system with 2 cores + 1 (master core).

Foreground task	LLC-load-misses	LLC-store-misses	uncore_imc/data_reads/(MiB)	uncore_imc/data_writes/(MiB)	IPC
619.lbm_s (Run Alone)	475,501,764	959,591,246	101,478.39	74,475.04	0.50
619.lbm_s (Co-scheduled)	217,591,827	435,743,459	79,543.03	55,447.04	0.57

Section F. Evaluations on 4-Core Setup

In this section, we compare three scenarios — no regulation, Memguard regulation, and LMS-AR Regulation — across few of SPEC CPU2017 workloads in a four-core co-scheduled setup. In order to better comprehend the interference effects and minimize the effects of cache on performance, in the first evaluation (Fig. 4) we choose workloads such that memory bandwidth required by the workload exceeds the last level cache size of the platform. The workloads considered here — 619.lbm_s, 519.lbm_r, 621.wrf_s, 521.wrf_r — are classified as highly memory intensive. We consistently observe this behavior of LMS-AR regulating efficiently with other memory intensive workloads as well. This aligns with our initial premise that an adaptive distribution of the memory bandwidth leads to lower interference.

In the second experiment, we choose workloads classified as low memory intensity for co-execution. In Fig. 5, clearly the regulation mechanism has little impact on the performance as workloads tend to operate very close to the sustainable bandwidth (slowdown ratio close to 1) and the allocations satisfy their smaller bandwidth requirement.

In a diversified workload setting (Fig. 6) with — 603.bwaves_s (H), 541.leela_r (L), 544.nab_r (L) and 620.omnetpp_s (M), we find that LMS performs worse for medium intensity workload, attributed to weights converging on a regulation budget that is too restrictive for this workload's bursty memory behavior. There is limited observation from this work that a computationally involved model (such as the LMS) for predicting bandwidth requirement could produce better estimates, which helps to allocate the right amount of bandwidth during each regulation period and improves over time. Additionally, as the core count increases in the system, the workloads will continue to benefit from this solution.

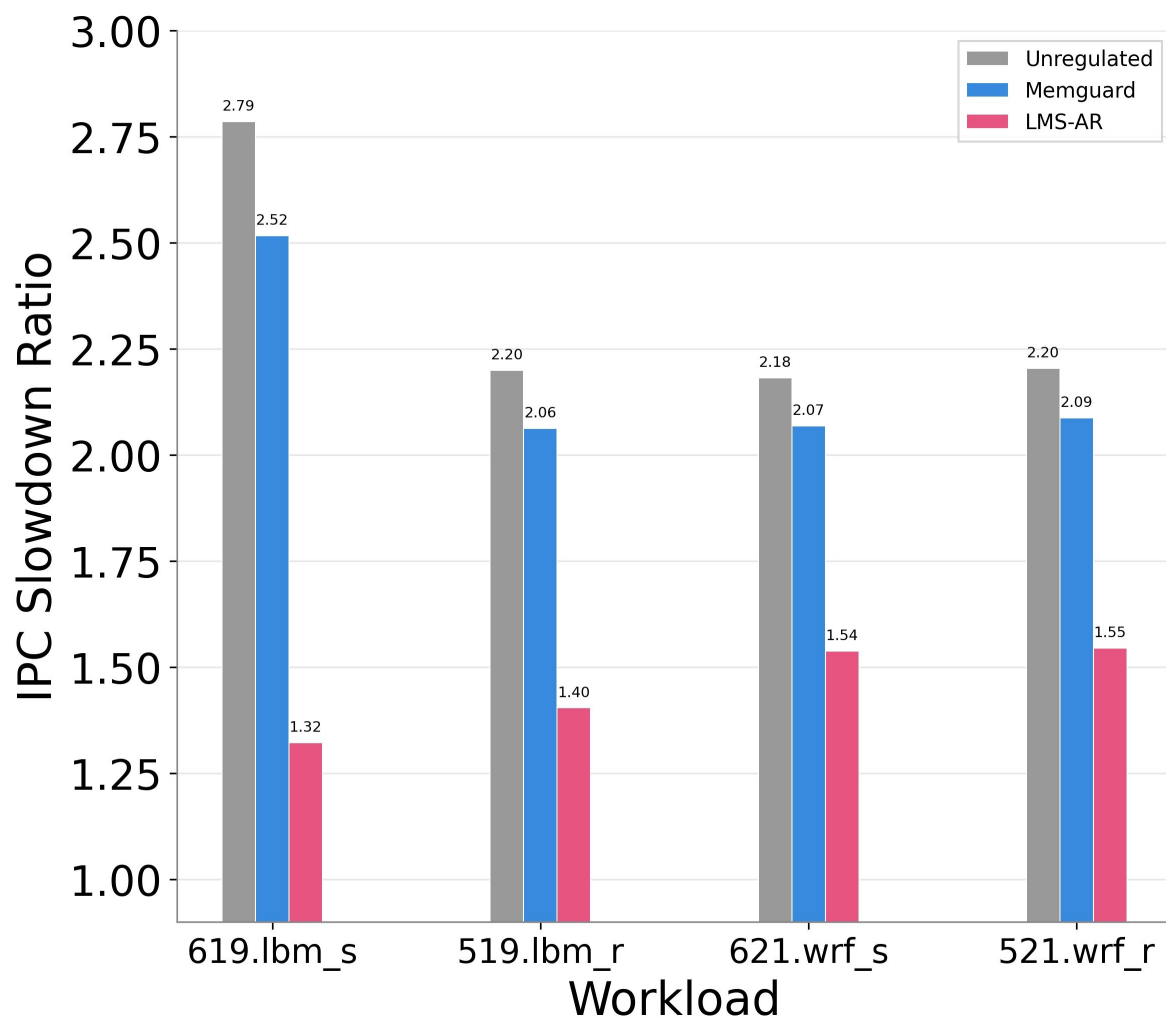
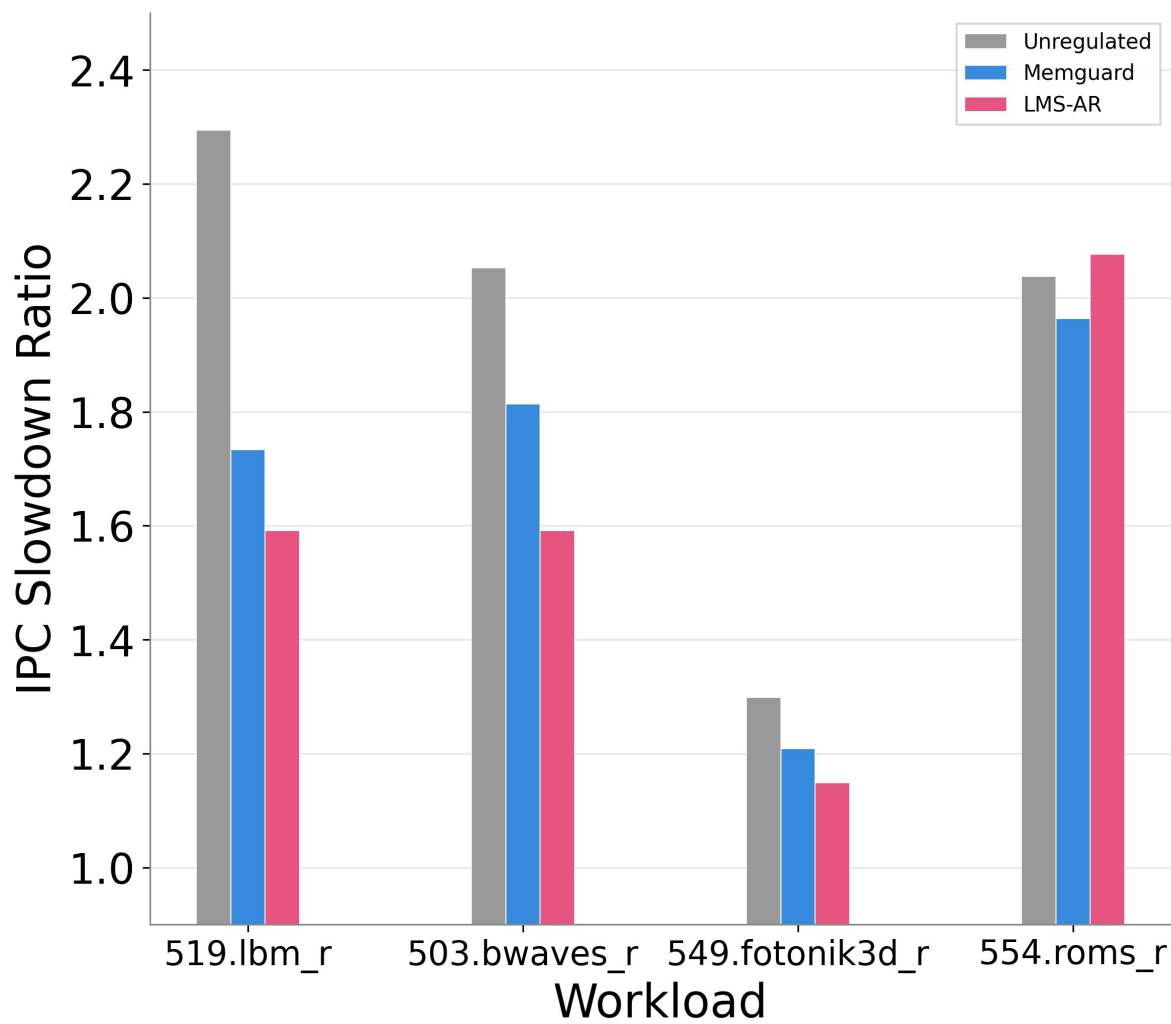


Figure 4. Workload Performance, 4 cores + 1 master core setup with high memory intensive workloads (619.lbm_s, 519.lbm_r, 621.wrf_s, 521.wrf_r).



Supplementary: Workload Performance, 4 cores + 1 master core setup with additional high-intensity workloads (519.lbm_r, 503.bwaves_r, 549.fotonik3d_r, 554.roms_r), illustrating that LMS-AR consistently regulates efficiently across other memory-intensive workloads.

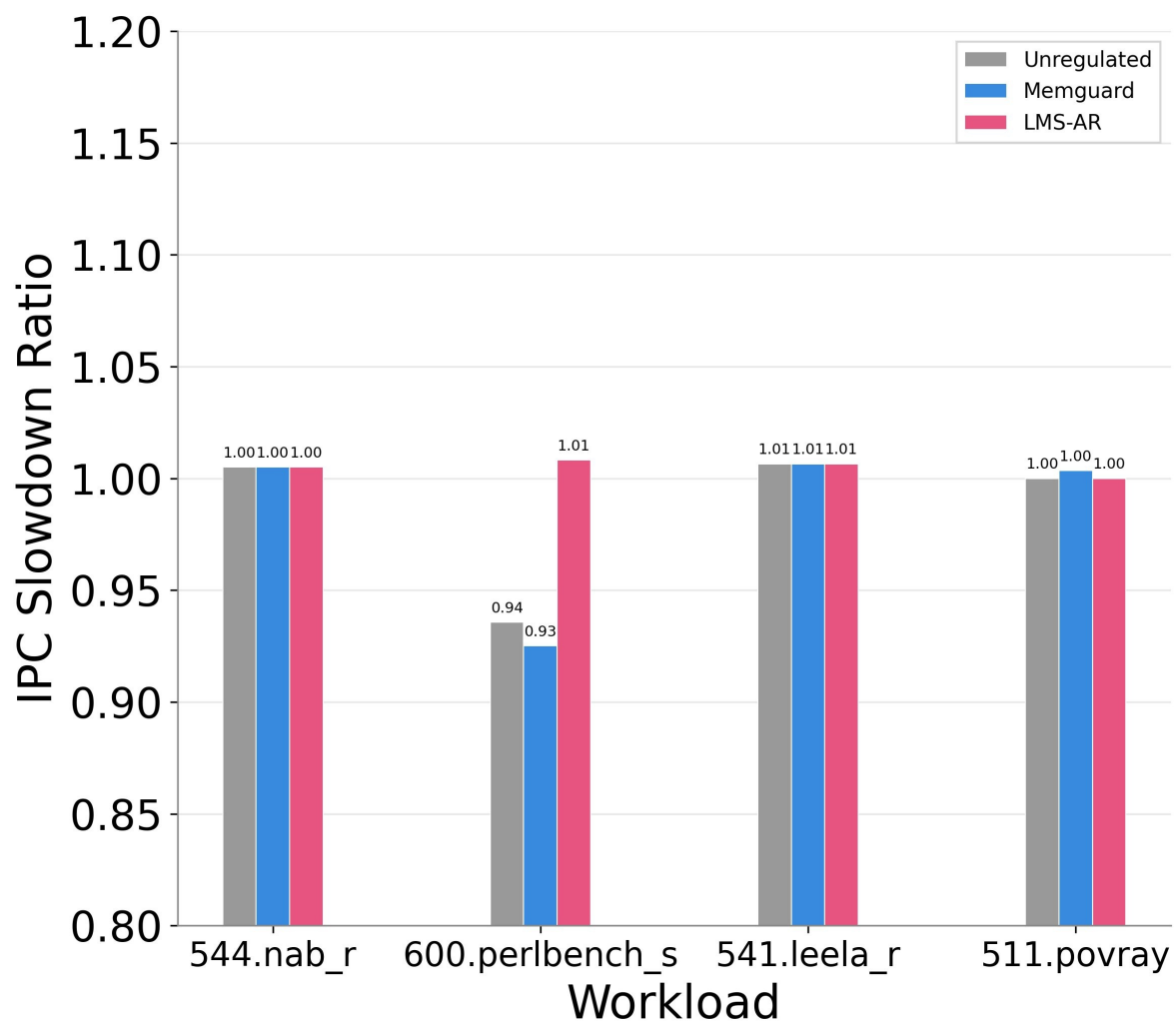


Figure 5. Workload Performance, 4 cores + 1 master core setup with low memory intensive workloads.

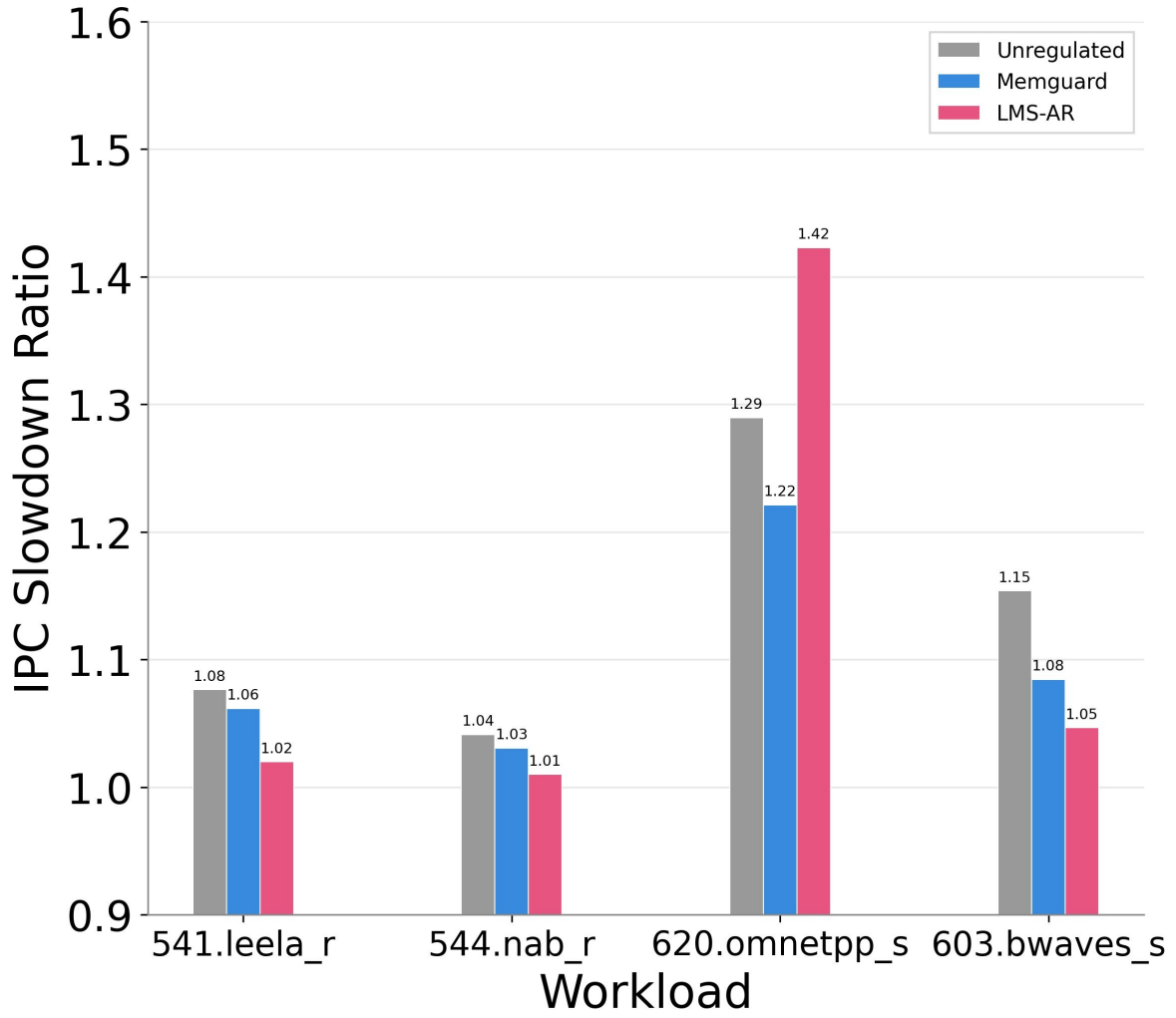


Figure 6. Workload Performance, 4 cores + 1 master core setup with diverse set of workloads (603.bwaves_s (H), 541.leela_r (L), 544.nab_r (L), 620.omnetpp_s (M)).

Section G. Impact of Regulation Interval

The regulation interval defines the fixed temporal period between successive invocations of the bandwidth control loop. Within each interval, cores execute and consume bandwidth according to their allocated budgets. Upon interval completion, actual bandwidth consumption is measured and new allocations are computed for the next interval. In our implementation, a master thread wakes periodically at fixed intervals to perform these measurements and allocation operations.

Regulation interval decides effectiveness of regulation mechanism, representing a trade-off between responsiveness and overhead which involves finding the regulation interval value that requires balancing two competing concerns. A long regulation interval can lead to poor estimates or delayed adaptation, leading to under allocation. A very small regulation interval can lead to excessive overhead and insufficient observation time, that captures random fluctuations instead of real workload characteristics or pattern.

The variation in IPC values for each workload, when co-scheduled on the 4-core setup, is illustrated in Fig. 7. Lower memory-intensive workloads (538.imagick and 544.nab) benefit most with increasing regulation interval, as longer regulation interval allows for a longer execution period. Workloads with higher memory requirement (such as 519.lbm) essentially maintain the same performance. At lower (less than 500 μ s) intervals, the advantage diminishes as frequent intervention to the control loop leads to overheads and causes the prediction mechanism to underperform.

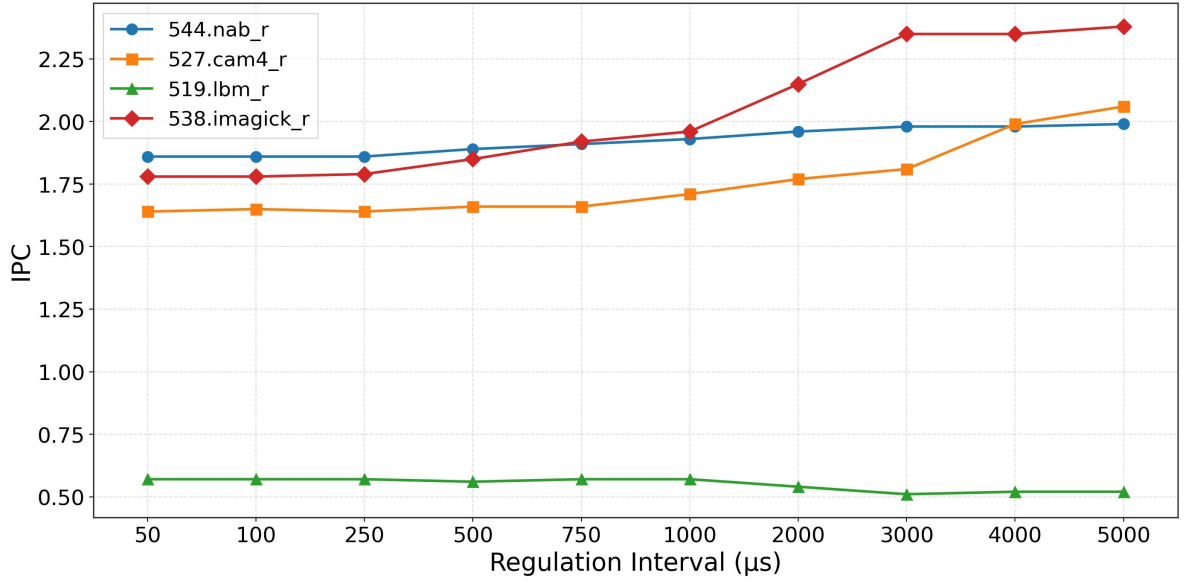


Figure 7. Variation in IPC performance of workloads in 4 + 1 core setup with varying regulation intervals (50 μs to 5 ms).

V. RELATED WORK

The problem of temporal isolation with main memory as the key contributor for interference in mixed critical systems has been studied and analyzed extensively [10], [11]. The solutions available in this space range between hardware design solution to software assisted solutions.

The hardware-assisted solutions include 1) enforcement hardware [12], [13], which virtually eliminates the overhead limits of interrupt-based throttling to achieve very fine-timescale bandwidth accounting, and 2) memory-controller hardware designs [14]–[18] that provide flexible priority-based controller designs with improvement on TDM based designs for performance and predictability 3) vendor-driven hardware QoS mechanisms Arm MPAM [19].

Software-assisted DRAM regulation mostly involves source-throttling mechanisms implemented in the OS or hypervisor, where each core/VM is given a memory budget over a fixed period and is paused when it exceeds that budget. The foundational work MemGuard and follow-up [2], [20] have been integrated into schedulability and mixed-criticality analysis [21], [22]. E-Warp [4] uses [2] to regulate the CPU core, but also extends to regulate traffic from an accelerator. Mempo [3], on the other hand, uses an out-of-core approach to regulate the cores, to reduce the memory transactions generated by per-core interrupt handlers in Memguard, enabling microsecond-scale on-off control with sliding-window policing and redistribution of unused bandwidth without disturbing the application cores with frequent interrupts, were also one of the motivations in our work. The work in [23] shifts monitoring closer to the true DRAM bottleneck by using controller-provided utilization counters inside a feedback loop, rather than relying solely on PMU. However this is applicable to specific hardware platforms that can support profiling statistics of the DRAM controller.

LMS-AR closely relates to Memguard in that both are 1) Linux kernel modules that leverage hardware PMCs to regulate shared resource usage 2) share the design philosophy of decomposing resource allocation into a guaranteed component for real-time tasks and a best-effort component for non-critical workloads 3) employs reclaiming mechanisms to minimize resource wastage from static partitioning. However, the two approaches differ significantly in their prediction and regulation strategies. LMS-AR incorporates a Least Mean Squares (LMS) adaptive filtering technique for prediction, to minimize the error between predicted and observed resource demand in real time, enabling more accurate tracking of workload and faster convergence. We do not compare with Mempo, though our work has similarities (an out-of-core design), as the master core that regulates other cores shares the same DRAM as the regulated cores and the platform lacks vendor-specific debug interface. However, the design enables exploring sophisticated prediction models with little impact on regulation performance.

VI. CONCLUSION

In this paper, we presented LMS-AR, a Linux kernel module for adaptive resource regulation on multicore platforms. It employs model-based control with real-time performance monitoring to dynamically adjust

resource allocation based on workload characteristics. Experimental results demonstrate that LMS-AR improves average-case performance for real-time tasks co-executing with best-effort workloads. These have shown to improve the performance of most of the low, medium and high intensity workloads. However, we will explore the efficiency of learning based prediction models in future work along with extending the framework to support heterogeneous computing platforms.

REFERENCES

- [1] B. Akesson, K. Goossens, and M. Ringhofer, "Predator: a predictable sdram memory controller," in *Proceedings of the 5th IEEE/ACM International Conference on Hardware/Software Codesign and System Synthesis*, ser. CODES+ISSS '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 251–256. [Online]. Available: <https://doi.org/10.1145/1289816.1289877>
- [2] "Memguard: Memory bandwidth reservation system for efficient performance isolation in multi-core platforms," in *Real-Time Technology and Applications - Proceedings*, 2013.
- [3] A. Zuepke, A. Bastoni, W. Chen, M. Caccamo, and R. Mancuso, "Mempol: polling-based microsecond-scale per-core memory bandwidth regulation," *Real-Time Systems*, vol. 60, pp. 369–412, 9 2024.
- [4] P. Sohal, R. Tabish, U. Drepper, and R. Mancuso, "E-warp: A system-wide framework for memory bandwidth profiling and management," in *2020 IEEE Real-Time Systems Symposium (RTSS)*, 2020, pp. 345–357.
- [5] B. Widrow, M. A. Lehr, F. Beaufays, E. A. Wan, and M. Bilello, "Learning algorithms for adaptive processing and control," *IEEE International Conference on Neural Networks*, pp. 1–8 vol. 1, 1993. [Online]. Available: <https://api.semanticscholar.org/CorpusID:17246524>
- [6] The Linux Kernel Project. (2026) Floating point api. [Online]; Accessed: March 19, 2026. [Online]. Available: <https://docs.kernel.org/core-api/floating-point.html>
- [7] Intel, "Intel Intel® 64 and IA-32 architectures software developer's manual." [Online]. Available: <https://www.intel.com/content/dam/develop/external/us/en/documents/335592-sdm-vol-4.pdf>
- [8] "Open suse documentation," [Online]; Accessed: May 19, 2026. [Online]. Available: <https://documentation.suse.com/sle-rt/15-SP6/html/SLE-RT-all/cha-shielding-cpuset.html>
- [9] Technical report on LMS-AR memory bandwidth regulator. [Online]; Accessed: May 19, 2026. [Online]. Available: <https://github.com/ss22ongithub/AdaptiveRegulator/wiki/Technical-Report-on-LMS%E2%80%9090AR>
- [10] T. Lugo, S. Lozano, J. Fernandez, and J. Carretero, "A survey of techniques for reducing interference in real-time applications on multicore platforms," *IEEE Access*, vol. 10, pp. 21 853–21 882, 2022.
- [11] M. Zini, G. Cicero, D. Casini, and A. Biondi, "Profiling and controlling i/o-related memory contention in cots heterogeneous platforms," *Software: Practice and Experience*, vol. 52, no. 5, pp. 1095–1113, 2022.
- [12] F. Farshchi, Q. Huang, and H. Yun, "Bru: Bandwidth regulation unit for real-time multicore processors," in *2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2020, pp. 364–375.
- [13] I. Izhbirdeev, D. Hoornaert, W. Chen, A. Zuepke, Y. Hammad, M. Caccamo, and R. Mancuso, "Coherence-aided memory bandwidth regulation," in *2024 IEEE Real-Time Systems Symposium (RTSS)*, 2024, pp. 322–335.
- [14] R. Mirosanlou, M. Hassan, and R. Pellizzoni, "Duomc: Tight dram latency bounds with shared banks and near-cots performance," in *Proceedings of the International Symposium on Memory Systems*, ser. MEMSYS '21. New York, NY, USA: Association for Computing Machinery, 2022.
- [15] F. Hebbache, M. Jan, F. Brandner, and L. Pautet, "Shedding the shackles of time-division multiplexing," in *2018 IEEE Real-Time Systems Symposium (RTSS)*, 2018, pp. 456–468.
- [16] R. Mirosanlou, M. Hassan, and R. Pellizzoni, "Drambulism: Balancing performance and predictability through dynamic pipelining," in *2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2020, pp. 82–94.
- [17] Y. Li, B. Akesson, and K. Goossens, "Architecture and analysis of a dynamically-scheduled real-time memory controller," *Real-Time Systems*, vol. 52, no. 5, pp. 675–729, 2016. [Online]. Available: <https://doi.org/10.1007/s11241-015-9235-y>
- [18] H. Kim, D. Broman, E. A. Lee, M. Zimmer, A. Shrivastava, and J. Oh, "A predictable and command-level priority-based dram controller for mixed-criticality systems," in *21st IEEE Real-Time and Embedded Technology and Applications Symposium*, 2015, pp. 317–326.
- [19] M. Zini, D. Casini, and A. Biondi, "Analyzing arm's mpam from the perspective of time predictability," *IEEE Transactions on Computers*, vol. 72, no. 1, pp. 168–182, 2023.
- [20] H. Yun, G. Yao, R. Pellizzoni, M. Caccamo, and L. Sha, "Memory bandwidth management for efficient performance isolation in multi-core platforms," *IEEE Transactions on Computers*, vol. 65, no. 2, pp. 562–576, 2016.
- [21] M. A. Awan, K. Bletsas, P. F. Souto, B. Akesson, and E. Tovar, "Mixed-criticality scheduling with dynamic memory bandwidth regulation," in *2018 IEEE 24th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2018, pp. 111–117.
- [22] G. Yao, H. Yun, Z. P. Wu, R. Pellizzoni, M. Caccamo, and L. Sha, "Schedulability analysis for memory bandwidth regulated multicore real-time systems," *IEEE Transactions on Computers*, vol. 65, no. 2, pp. 601–614, 2016.
- [23] A. Saeed, D. Dasari, D. Ziegenbein, V. Rajasekaran, F. Rehm, M. Pressler, A. Hamann, D. Mueller-Gritschneider, A. Gerstlauer, and U. Schlichtmann, "Memory utilization-based dynamic bandwidth regulation for temporal isolation in multi-

cores," in *2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2022.